

PERFORMANCE COMPARISON OF CNN INFERENCE ON MNIST: C++ VS. PYTORCH

Ari Hölscher
hantje@ethz.ch

Matteo Meciani
mmeciani@ethz.ch

Nicola Rupf
nrupf@ethz.ch

ABSTRACT

How much of a general-purpose framework’s performance can a specialized implementation recover, and at what cost to generality and portability? We compare PyTorch against a ladder of nine C++ forward passes for a three-block MNIST classifier, on one Arm Neoverse V2 core, ranging from a naive loop to a NEON intrinsics kernel. Portable optimizations give a $2.75\times$ speedup over our naive baseline, shape specialization adds $1.6\times$, and NEON, the only non-portable step, adds $3.06\times$ more to reach $13.46\times$, within 3% of PyTorch. An im2col/GEMM recast reaches $7.88\times$ while keeping the code portable. Our code is available in a dedicated [GitHub repository](#).

1 INTRODUCTION

When characteristics of data in a local neighborhood are more important than where in an entire sample such a shape appears, convolutions with kernels, the way CNNs apply them, can be useful as they capture local relative relationships. For background, [1] gives an introduction to CNNs, and [2] made a great interactive [cnn-explainer website](#).

A recurring question in HPC is to what extent tasks are best handled by general purpose frameworks, which trade performance for versatility and ease of use, compared to specialized solutions that exploit domain knowledge for speed. This tension recurs at several levels: general libraries versus manually tuned implementations, general compilers versus compilers enhanced with domain knowledge [3], or general processors versus domain specific architectures [4]. Work on combining both approaches is active, in weather and climate research for instance through domain specific languages such as GT4Py [5] or multilevel compilers [6], which apply different degrees of specialization at different levels to suit varying requirements. Generally, when and to what degree to specialize, remains an interesting question.

CNN inference cost is dominated by convolutional layers, whose arithmetic intensity depends on problem size and hardware [7]. This mirrors stencil computations, common in weather and climate models and elsewhere [8, 9, 10], which apply a fixed operation at every grid point and are typically memory bound, even on CPUs [11]. A key difference is that CNN kernels vary in shape and weights across layers and applications, so hardcoding them, common for stencils such as formulas of discretized differential operators with fixed scalar factors, is less practical. Furthermore, the inputs to weather and climate model stencil operations are usually much larger than to the CNN we will investigate in this experiment. Of the usual stencil characteristics, low arithmetic intensity, predictable access patterns, and limited temporal data reuse [11], we expect CNNs to share the first two, while possibly offering somewhat higher tem-

poral data reuse per FLOP due to kernels also being treated as variables and not hardcoded.

We ask not only whether a hand-tuned implementation can match PyTorch – a strong, well-optimized baseline – but also at what level of specialization such gains appear, and at what cost to readability and replicability. We expect gains to be greater for small kernels and inputs, and thus focus on a fixed and relatively small problem.

2 EXPERIMENT DESIGN

2.1 EXPERIMENT OVERVIEW

Our chosen problem is the classic single-image MNIST classification with a CNN. We implement a framework that allows us to verify and time C++ implementations of the forward pass of a fixed CNN architecture against the PyTorch implementation of the same network. Performance is always measured on a single CPU core on the [Santis cluster](#) of [CSCS’s Alps](#) (see §3.5). Starting at a naive and general baseline implementation, our comparison covers increasing levels of code optimization and specialization towards the specific CNN architecture and hardware. This scope was chosen to fit the course’s allocated effort, and Section 6 outlines how the comparison could be extended to additional cases and parameters.

2.2 CNN ARCHITECTURE

The network is a conventional classifier of the kind introduced by [12] and used as a standard teaching example. We did not adopt it from a specific published model; it was chosen because it is small, representative of the convolution shapes that dominate CNN inference cost, and simple enough that a faithful C++ reimplementation is tractable within the project’s scope.

The architecture consists of three convolutional blocks with output depths 32, 64 and 128 respectively, each followed by a ReLU activation function. The first two

Layer	FLOPs
Conv2d (1, 32, 3)	389 376
MaxPool2d (2, 2)	–
Conv2d (32, 64, 3)	4 460 544
MaxPool2d (2, 2)	–
Conv2d (64, 128, 3)	1 327 104
AvgPool2d (1)	–
Linear (128, 10)	2 560
Total (≈ 6.18 MFLOP)	6 179 584

Table 1: Arithmetic cost per sample. The ReLU and Flatten layers are omitted, as their cost is negligible compared to the convolutions.

blocks use max pooling; the third uses global average pooling, followed by a linear classification head. No padding is added, all convolutional kernels have dimension 3×3 with stride 1, while the max-pool kernels all have dimension 2×2 and stride 2. The PyTorch implementation also requires a final flattening before the linear classification head, purely to match the expected input shape of `nn.Linear()`.

2.3 COST ANALYSIS

Table 1 lists the arithmetic cost of each layer, and two of its properties shape the rest of the report. First, the cost is extremely concentrated: the second convolution alone accounts for 72 % of all floating-point operations (FLOPs) and the three convolutions for 99.96 %, which is why our optimization effort targets `conv2d.forward` almost exclusively. Second, the problem is small. One forward pass costs approximately 6.18 MFLOP. The parameters occupy 367 KiB and one sample’s activations occupy a further 150 KiB, so at batch size 1 the whole 517 KiB working set fits in the private L2 cache of one core (see subsection 2.7). This is a regime limited by instruction throughput rather than memory bandwidth, and the one in which we expect specialization to pay off most.

2.4 DATA, TRAINING, AND WEIGHTS

We download the MNIST dataset via [13]. Its rich history is reconstructed by [14]. Excluding the batch dimension and given the $1 \times 28 \times 28$ input dimensions of MNIST samples [14], the intermediate tensor shapes are thus $32 \times 26 \times 26 \rightarrow 32 \times 13 \times 13 \rightarrow 64 \times 11 \times 11 \rightarrow 64 \times 5 \times 5 \rightarrow 128 \times 3 \times 3 \rightarrow 128$, before the final linear layer maps to the ten output classes.

We train the reference PyTorch model with k -fold cross-validation [15] (default $k = 5$): the pooled training and test splits of MNIST are partitioned into k folds, and for each fold a freshly initialized network is trained on the remaining $k - 1$ folds and evaluated on the held-out fold. Unlike the standard use of k -fold cross-validation for model selection [15], the folds instead serve to guard against a single unlucky training run and yield a usable set of weights. Training uses cross-entropy loss and the Adam optimizer [16] with base learning rate 10^{-4} ,

batch size 10, for 5 epochs per fold. The random seed is fixed at 0 for both PyTorch and NumPy, so training is reproducible up to the nondeterminism of the GPU kernels used for the backward pass. Consequently, no separate test set is held out, and the winning fold’s weights are exported directly, without retraining on the pooled data. The reported accuracy is therefore an optimistic estimate, not an unbiased measure of generalization performance. Since our goal is only to verify that the C++ implementation reproduces the PyTorch forward pass given identical weights, the model’s ground-truth prediction accuracy is irrelevant, but of course nice to have. For validation of the C++ performance, the weights of the best-performing fold are exported as raw `float32` binaries, one file per parameter tensor (i.e. each weight or bias tensor belonging to a layer of the model) in C-contiguous (row major) order.

Both implementations read the same MNIST IDX file directly and divide the unsigned byte pixel values by 255, with no further preprocessing, so the two forward passes see bit-identical inputs as well as bit-identical weights.

2.5 SOFTWARE

All measurements were taken on the Santis cluster [17] (aarch64, Linux 6.4, glibc 2.38), inside the CSCS user environment (`uenv`) `image pytorch/v2.9.1:v2` with the default view, which fixes the compiler and the PyTorch build for every job. Extra Python packages are installed into a virtual environment layered on top, since the `uenv` filesystem is read only.

2.5.1 C++ COMPILER

The C++ code is compiled with GCC 14.2.0 as packaged by Spack inside the above `uenv` image, invoked as `g++`. The optimization flags are

```
-std=c++17 -O3 -march=native -mcpu=native
-ftree-vectorize -fopenmp-simd -DNDEBUG
```

together with `-Wall -Wextra -Wpedantic`. Compilation happens on the compute node inside the job script, so `-march=native` and `-mcpu=native` resolve against the Grace cores that then run the benchmark rather than against the login node.

We deliberately do not enable `-ffast-math`. Doing so would let the compiler reassociate the reduction in the innermost convolution loop and vectorize it automatically, with no source changes at all. Since our goal is to attribute each performance gain to a specific, identifiable transformation, we exclude flags that would let the compiler silently perform optimizations on our behalf. Every reassociation reported here is therefore one we performed explicitly in the source.

2.6 PYTHON VERSION

The reference implementation uses PyTorch 2.9.1, the version pinned by the `uenv` image, on Python 3.12.12. Supporting scripts use NumPy, torchvision

for the dataset download, and scikit-learn for the cross-validation splits. We describe the actually used backend in subsection 4.1.1.

2.7 HARDWARE

We are running both the PyTorch and C++ implementations on the Santis cluster [17], consisting of NVIDIA GH200 chips that combine and connect a Grace CPU and a Hopper GPU with NVLink-C2C [18].

The weights are trained on the GPU, while all benchmarking is performed on a single CPU core. The Grace CPU provides 72 Arm Neoverse V2 cores, each with a private 64 KiB L1 data cache and a private 1 MiB L2 cache, backed by a large shared last level cache and LPDDR5X memory. The property that matters here is the vector unit: Neoverse V2 issues up to four 128-bit SIMD operations per cycle and supports NEON at a 128-bit vector length, giving a single-precision peak of $4 \times 4 \times 2 = 32$ FLOPs per cycle per core when every operation is a fused multiply-add [19].

Given the 32 FMA operations per cycle, at its nominal clock frequency of 3.3 GHz, each core of the Grace CPU has a theoretical peak of approximately 105.6 GFLOP/s per core [19].

3 METHODOLOGY

3.1 EVALUATION METRICS

Performance was evaluated by comparing the wall-clock runtime (in seconds) of each implementation against PyTorch. We additionally report median CPU cycle counts and the achieved performance in GFLOP/s, obtained by dividing the 6.18 MFLOP of a single sample (Table 1) by the measured time. The absolute figure is what allows a comparison against the machine’s single-core peak of roughly 105.6 GFLOP/s, and therefore tells us how much of the available performance each level of specialization actually recovers, which a speedup ratio alone cannot.

Cycle counts are available for the C++ implementations but obtaining a precise number for the PyTorch implementation proved to be challenging, since a counter started and stopped from Python also measures the interpreter overhead around the call. C++ and PyTorch are therefore compared on wall clock time only, and cycles are used exclusively among the C++ variants, where the instrumentation is identical.

3.2 TIMING INSTRUMENTATION

A lightweight timing class using the `clock_gettime` function was implemented to keep track of the execution times, using the `CLOCK_MONOTONIC_RAW` clock. Cycle counts come from the Linux performance monitoring unit: a `PERF_COUNT_HW_CPU_CYCLES` counter opened through `perf_event_open` with `exclude_kernel` and `exclude_hv` set, so that only cycles retired in user space on behalf of our process are counted. To further overcome hardware performance fluctuation and possible

noise we report medians rather than single measurements.

A second concern is that at -O3 the compiler may observe that the result of a forward pass is never read and delete the entire computation. Every run therefore accumulates a checksum over the final logits into a volatile global, which forces the store to be emitted and hence the logits to be computed. Its final value is printed as evidence that the guard was active.

3.3 STATISTICS AND CACHE STATE

In order to obtain a reliable and reproducible runtime measurement, the benchmarking phase starts with a warmup of 10 iterations, so that the hardware frequency reaches its steady-state frequency before the timed runs. After the warmup, the runtime is measured 30 times per implementation, with the median time being reported to filter out noise due to background processes and in order to obtain a reproducible number.

The median is preferred to the mean because the distribution is not symmetric: the achievable time is bounded from below by the work the kernel must do, while occasional interference from the operating system produces a long right tail that would pull a mean.

All measurements are taken with a warm cache. Every buffer is allocated once before timing begins and reused across all runs and implementations, so after the warmup the entire 517 KiB working set of a batch-size-1 pass is resident in the executing core’s private L2.

3.4 VALIDATION

To validate the results, we implemented checks that can be toggled on to verify the output’s correctness. For each C++ implementation, we compare the 10 logits produced by the network’s last layer for each dataset sample, and verify that they match with the PyTorch ones. The comparison is executed with NumPy’s `allclose`, with the absolute tolerance set to 10^{-4} and the relative tolerance left at its default of 10^{-5} ; both are reasonable values to account for the different order of floating point operations in the two implementations. The tolerance itself is a statement about associativity rather than accuracy: every optimization in Section 4 changes the order in which products are summed, and since we compile without `-ffast-math`, that deliberate reassociation is the only source of divergence.

The check runs over the entire 10 000-image test split, not a single sample, and two criteria must both hold: the logits must agree within the tolerance, and the predicted class must agree for every image. The second matters because a tolerance on the logits alone would not catch a systematic error that fell just inside it yet flipped a borderline classification.

3.5 EXPERIMENTAL CONTROLS

The main parameters that were varied during the experiments were the number of timed runs to extract the median out of and the number of warmup runs. The val-

ues we ultimately landed on (30 and 10) were chosen after confirming the times obtained were reproducible and not suffering from hardware background noise. Beyond the timing protocol, we ruled out several accidental sources of error.

Single core, single thread. Jobs request one CPU per task and bind it explicitly, and `OMP_NUM_THREADS` is set to 1. On the PyTorch side, `torch.set_num_threads(1)` additionally prevents the ATen intra-op thread pool from spreading a single convolution across cores, which it does by default whenever the problem is large enough. The `#pragma omp simd` directive used in the `im2col` variant requests vectorization only and does not spawn threads.

CPU, not GPU. The nodes carry a Hopper GPU that PyTorch would use if a tensor were placed on it. The benchmark never calls `.to(device)`, so the comparison is CPU against CPU. The GPU is used only for training, outside the timed path.

Single Inference only. `model.eval()` and `torch.no_grad()` prevent PyTorch from allocating and populating the autograd graph during the forward pass, work the C++ implementation does not perform. Furthermore, we note that unless stated otherwise, our C++ implementations are evaluated at a batch size of 1, where we expect most gains compared to PyTorch.

4 CNN IMPLEMENTATIONS

4.1 PYTORCH

In the following section, we define the implementation of the CNN forward pass in Python, using the heavily optimized PyTorch library, created by Facebook AI Research, now governed by the Linux Foundation, see [20]. Python is an interpreted programming language and is broadly considered too slow for HPC. Deep Learning libraries such as PyTorch use it as an API or "wrapper" to call pre-compiled binaries written in C/C++ or CUDA, see [21].

PyTorch's CPU operations are typically routed through performance-optimized backend libraries such as Intel MKL (Math Kernel Library), OpenBLAS, or oneDNN (formerly MKL-DNN). Which of these is actually invoked for a given operation is not visible from Python code alone, we analyze this for our specific build in subsection 4.1.1.

Making the comparison fair, as set out in subsection 3.5, requires overriding two further PyTorch defaults that are tuned for training rather than single-threaded inference. By default, PyTorch spawns as many threads as the node exposes, which would make its runtime impossible to compare directly against a single-threaded C++ implementation; we therefore restrict it to a single CPU thread via `torch.set_num_threads(1)`. Without an explicit inference mode, PyTorch would also build a computational graph in memory to trace the

chain rule for the backward pass, see [22], work the forward-pass-only C++ implementation never performs; setting `model.eval()` and wrapping the call in a `torch.no_grad()` context disables this tracking and enforces pure computation.

The code for the forward pass loads binary weights from pre-training, which was performed in another script, through a custom `load_bin` function. It is necessary to provide both the PyTorch and the C++ implementations with the same binary weights from pre-training to ensure comparability. Otherwise, PyTorch, for one, would initialize the model's weights randomly.

In order to transition from the 4D output of `AdaptiveAvgPool2d` to the 2D input expected by `nn.Linear`, the implementation requires an explicit `nn.Flatten()` layer.

4.1.1 BACKEND

Determining which of the backend libraries mentioned above is actually invoked for a given operation requires profiling: `torch.profiler` traces calls down to the level of ATen [23], PyTorch's general-purpose C++ tensor and operator library that dispatches to CPU, CUDA, or specialized kernels depending on what is available. `torch._config.show()` reports `BLAS_INFO=open`, confirming OpenBLAS rather than Intel MKL is linked in this build, and `USE_MKLDNN=OFF`, meaning the oneDNN backend is not compiled in at all.

Profiling our forward pass confirms the practical consequence: all three Conv2d layers (15 calls total) are executed through `aten::_slow_conv2d_forward`, accounting for 56.10% of self CPU time and 62.81% of total CPU time, i.e. every convolution falls back to ATen's generic kernel rather than being routed through oneDNN or a hardware-tuned path. A likely explanation is that the disabled `USE_MKLDNN` flag is responsible for this fallback, since oneDNN with the Arm Compute Library (ACL) is the backend that would normally accelerate convolutions on aarch64. We have not independently confirmed this is the sole cause, but the observation and the disabled flag are at least consistent with each other.

This configuration is inherited from the pre-built PyTorch binary provided by CSCS's `pytorch/v2.9.1:v2_uenv`. It was fixed when that binary was built, not something adjustable from our own code, so changing it would mean building PyTorch from scratch rather than using the shared system build.

4.2 C++ IMPLEMENTATION

4.2.1 OPTIMIZATION ROADMAP

As laid out in subsection 2.3, the three convolutions dominate the arithmetic cost and our optimization efforts thus all target the convolution kernel, with one exception noted at the end. The ReLU, pooling and linear layers share implementations, so unless stated otherwise, any difference in measured time is attributable to the convolution alone.

The implementations do not form a single chain. The blocked baseline and the im2col variant are side branches that start again from the baseline, while the remaining six form a cumulative ladder in which each step keeps everything the previous step did and adds one transformation:

```
baseline → restructured → hoist+restrict
          → reorder → specialized → NEON blocked
          → specialized maxpool
```

Reading the results table as a linear progression would therefore be wrong for the two side branches, and we label them as such throughout. The ladder is ordered by decreasing generality: the first four steps preserve the baseline’s full generality, the fifth gives up arbitrary stride and padding, and the sixth gives up portability across instruction sets. That ordering is deliberate, since the question the report asks is at what point along it the gains appear.

4.2.2 BASELINE

With the aim in mind to mirror the preceding PyTorch implementation, certain architectural decisions needed to be made for the C++ forward pass.

For the memory layout, rather than using complex multi-dimensional objects, we chose to store all intermediate activations and weights as flat contiguous 1D arrays using `std::vector<float>` within a custom Tensor struct. The struct tracks the logical dimensions of batches, channels, height and width of the data.

Because 1D arrays were chosen, no explicit flattening layer was necessary to mirror the Python code for this step.

In the baseline implementation, the convolutional layer of the CNN is implemented in `conv2d_forward` as a deep, 7-level nested loop structure that iterates over the batch, output channels, output spatial dimensions height and width, input channels and kernel dimensions.

Other layers within the CNN are also implemented as straightforward loops, such as the ReLU layer, pooling layer and linear layer. The ReLU layer `relu_forward` is clamping negative vector values to 0 in an element-wise fashion, while the baseline pooling layer is implemented as an intricate nested loop in `maxpool2d_forward` and designed to compute windowed maxima. The final linear layer `linear_forward` is performing looped matrix multiplications of weight and input and adding the bias.

Since memory is flat, matrix multiplication is only conceptually traced and every pixel access to the MNIST images requires manual stride arithmetic, for instance:

```
int in_idx = b * (input.channels
    * input.height * input.width)
    + ic * (input.height * input.width)
    + ih * input.width + iw;
```

The innermost loop contains boundary checks to resolve padding regions and act as virtual zero-padding, for instance:

```
if (ih >= 0 && ih < input.height
    && iw >= 0 && iw < input.width)
```

4.2.3 RESTRUCTURED NESTED LOOPS

The cumulative ladder begins by removing the branch from the innermost loop without giving up any generality.

The observation is that the baseline’s boundary condition does not depend on the input channel or on the accumulated value, only on the output position and the kernel offset, both known before the innermost loop starts. Rather than testing every element, `conv2d_forward_restructured` solves the condition once per output position for the range of kernel offsets that land inside the input:

```
int c_h = oh * stride - padding;
int kh_lo = std::max(0, -c_h);
int kh_hi = std::min(ker_h, input_h - c_h);
```

and analogously for columns. The kernel loops then iterate over `[kh_lo, kh_hi]` and `[kw_lo, kw_hi]`, every iteration is valid by construction, and the innermost body reduces to two index computations, two loads and one multiply-add.

This is a well-known transformation, splitting the iteration space into a boundary region and an interior one, and it is exactly what a stencil code does to keep conditionals out of its hot loop. For our network the padding is zero, so the computed bounds always evaluate to the full kernel extent: the transformation costs a few loop-invariant integer operations per output pixel and saves a branch per multiply-add. Generality is fully preserved. The variant additionally copies the Tensor dimension fields into locals, which looks cosmetic but matters for the same reason the next step does: reading `input.height` inside a loop is a load through a reference that the compiler cannot always prove a store did not modify.

4.2.4 HOISTED INDICES AND restrict

The next step addresses that last point directly. `conv2d_forward_hoist_restrict` extracts raw pointers such as `const float* _restrict input_ptr` from the underlying vectors. The annotation promises the compiler that these regions do not overlap; without it, a store through the output pointer could in principle alias the input or the weights, forcing a reload after every store and blocking the reordering of loads across stores. The variant also does the index arithmetic by hand, pre-computing the stride products once per call and the partial offsets at each loop level, so the innermost index is an addition of terms invariant at that depth rather than a chain of multiplications. This is manual strength reduction.

We report the step separately because its outcome is instructive: the improvement is small. GCC 14 at -O3 already performs most of this strength reduction on the restructured code, and the accumulator was a local scalar and therefore already register-resident, so the aliasing guarantee has little left to unlock. Hand-optimizing

what the compiler already does reliably buys nearly nothing, and the real barrier lies in the shape of the innermost loop, which the next step changes.

4.2.5 REORDERED NESTED LOOPS

Up to this point every variant on the ladder accumulates one output pixel at a time into a scalar. That is a reduction, and a floating point reduction cannot be vectorized without changing the order of the additions, which the compiler may not do under strict IEEE semantics and which we did not authorize by enabling `-ffast-math`. No further cleanup inside that loop can vectorize it.

`conv2d_forward_reorder` changes the loop nest so the innermost loop runs over output columns instead of kernel offsets. A row accumulator `std::vector<float> acc(output_w)`, initialized to the bias, holds one full output row, and the innermost loop becomes

```
for(int ow = ow_lo; ow < ow_hi; ++ow) {
    int iw = ow * stride - padding + kw;
    acc_ptr[ow]
        += input_ptr[in_row_offset + iw]
           * w_val;
}
```

with `w_val` loop-invariant. Successive iterations write to distinct elements, so there is no cross-iteration dependence and the loop is a scaled vector addition the compiler vectorizes directly. The row is written out once at the end.

Preserving generality requires solving the boundary condition in the other direction: where the restructured variant asked which kernel offsets are valid for a fixed output position, this one must ask which output columns are valid for a fixed kernel offset. The resulting `ow_lo` and `ow_hi` again keep the bounds outside the innermost loop.

This is the same structure the `im2col` GEMM reaches, which is why the two land in a similar regime by different means; the difference is that `im2col` pays with a $K \times N$ materialized buffer, whereas this variant pays only for one row of accumulators and reads the input in place. Note also that the input access is unit-stride only when `stride` is 1. The loop stays correct for larger strides but degenerates into a gather, so the quality of the generated code, though not its correctness, already depends on the parameters of our particular network.

4.2.6 SPECIALIZED IMPLEMENTATION

The previous variant is the last drop-in replacement for a general convolution. `conv2d_forward_specialized` takes the step the report is really about and bakes the network's parameters into the code, visibly so: the signature no longer accepts `stride` and `padding` at all.

With `stride` fixed to 1 and `padding` to 0, the input row index collapses to `ih = oh + kh`, all four boundary computations disappear because every kernel offset is valid for every output position, and the innermost loop runs over the full range with a unit-stride access, `acc[ow] += input_ptr[in_row_offset + ow + kw]`

* `w_val`. The row accumulator also moves from a heap-allocated `std::vector` to a fixed `alignas(16) float acc[MAX_OW]` with `MAX_OW = 64`. That removes an allocation per call and tells the compiler the base address is 16-byte aligned, so it can emit aligned vector accesses without a scalar prologue.

The price is a validity domain the type system does not express and the code does not check. The kernel is correct only for `stride 1`, zero padding, and output widths up to 64; a wider layer would silently overflow the stack array. Our three convolutions have widths 26, 11 and 3, so the constraint holds with room to spare, but this is the first point on the ladder where using the kernel outside its intended configuration produces wrong answers rather than merely slow ones.

4.2.7 NEON BLOCKED IMPLEMENTATION

The final convolution variant is the fastest of the ladder by a wide margin, and it is also the one we did not write ourselves entirely. It stops relying on the compiler's autovectorizer and issues ARM NEON instructions explicitly through intrinsics, combined with the register blocking idea from subsection 4.2.9. Writing intrinsics by hand and tuning blocking factors against a specific register file is laborious and hardware-specific work, and we were more interested in what it buys than in performing it, so we produced this implementation with the assistance of a large language model (Claude Opus 4.8), giving it our existing kernels and the layer shapes as context. It then went through the same verification and timing framework as every other variant, with no exception made for it. Three ideas carry the speedup.

Weight packing. In the original `[oc][ic][kh][kw]` layout the weights consumed together sit $IC \times K_h \times K_w$ floats apart, so they are rewritten once into an order that makes them contiguous and readable by walking a pointer forward, as in the packing step of a tuned GEMM.

The microkernel. `conv_row_tile` computes several output channels of one output row at a time, keeping the whole tile of partial sums in vector registers and accumulating with the fused multiply-add `vfmq_n_f32`. Each input vector is loaded once and feeds one multiply-add per channel in the tile, and each weight scalar is reused across the row, so both operands are amortized instead of being re-read per multiply-add as in the baseline.

Choosing the blocking factors. AArch64 exposes 32 vector registers, and a tile that does not fit spills to the stack and gives the benefit straight back. A dispatcher sizes the tile from the output width, falling back to the scalar kernel of subsection 4.2.6 for rows too wide to block, and selects a template instantiation so the tile extents stay compile-time constants and the tile loops unroll completely. Our three layers use 14, 12 and 4 accumulator registers.

At the edges, the kernel reads a few elements past the end of each input row, so the driver pads the input buffer with zeroed slack and stores the last vector of each output row elementwise. It assumes the stride-1, zero-padding case, requires `arm_neon.h` and is tied to AArch64, so this is where the code stops being portable at all.

4.2.8 SPECIALIZED MAXPOOL2D

The last variant is the only one that changes a layer other than the convolution. It keeps the NEON blocked convolution and replaces `maxpool2d_forward` with `maxpool2d_forward_specialized`, which hard-codes the 2×2 window and stride 2, drops the bounds checks, and stages a row of maxima into an aligned stack buffer before writing it out.

It does not help. Its median runtime is statistically indistinguishable from the NEON blocked kernel it derives from: the two differ by well under a percent (Table 2), a gap within our run-to-run noise, so the specialization buys no measurable time. We report it because the reason is a useful check on the whole exercise. First, Amdahl’s law: the pooling layers perform no multiply-accumulates at all and together examine roughly 29 000 elements against 3.09 million multiply-accumulates in the convolutions, so once the convolutions are an order of magnitude faster, even removing pooling entirely would barely register. Second, the transformation is not clearly an improvement in isolation. The generic loop computes a maximum in a register and writes it straight out, whereas the specialized version writes to a staging buffer and then copies it, adding a pass over the data. The bounds checks it removes were loop-invariant in practice and perfectly predicted. Neither change pays for the added traffic.

Together with the small gain from the `restrict` step of subsection 4.2.4, this is the clearest evidence in our results that the profitable optimizations were the ones that changed the structure of the inner loop, and that effort spent elsewhere, however locally reasonable, did not transfer into runtime.

4.2.9 BLOCKED BASELINE

The first side branch attacks data reuse rather than loop overhead. In the baseline, each input pixel is loaded once per output channel it contributes to, so an input value is re-read 32, 64 or 128 times over a layer.

`conv2d_forward_blocked` introduces a two-dimensional register tile over output channels and output columns, with extents `OC_TILE` and `OW_TILE` both set to 4. The loop order becomes batch, output row, output-channel tile, output-column tile, and an accumulator array `float acc[OC_TILE][OW_TILE]` holds the sixteen partial results, initialized from the bias. Inside the tile, one input value is loaded once into `in_val` and multiplied against the four weights of the four output channels, so input loads per multiply-add fall by a factor of `OC_TILE`. Because the array is small and its indices become compile-time constants after unrolling, the compiler keeps the whole tile in registers.

The variant also lifts the row bounds check out of the innermost loop, testing `ih` once per kernel row, while the column check remains inside.

The tile extents were set to 4 by hand and not tuned; a proper search over both is one of the extensions listed in Section 6.

4.2.10 IM2COL IMPLEMENTATION

Since a convolution is a linear operation, it can be described as a matrix multiplication if the input tensor is first flattened into a vector (see for instance Chapter 9 in [24]).

There exists an implementation of the convolutional layer of a CNN that is based on this idea and was first popularized specifically in the context of High-Performance CNNs by [25], at the time at Microsoft.

The fundamental trade-off of this so-called `im2col` implementation is memory overhead for contiguous data access. This means that overlapping image patches that the kernel would operate on consecutively, not simultaneously, get unrolled into separate columns, requiring additional memory. The intricate stride math (see Sec. 4.2.2) during the calculation phase gets shifted to the memory allocation and writing to memory phase, which is described in the following, while the calculation happens in the form of a so-called General Matrix Multiplication, in short, GEMM.

For our forward pass with `im2col`, we reuse the very same ReLU, pooling and linear layer functions as the baseline implementation. Only the `conv2d_forward` calls are swapped for `conv2d_forward_im2col`. The latter introduces an intermediate memory buffer `im2col_buf`, which allocates memory for a matrix of size $K \times N$, where K equals the total elements in one kernel patch, given by $\text{Input Channels} \times \text{Kernel Height} \times \text{Kernel Width}$, for instance $1 \times (3 \times 3)$ for a 3×3 -pixel kernel operating on a grey-scale image. N denotes the total number of distinct spatial patches that the kernel could potentially operate on and is given by $\text{Output Height} \times \text{Output Width}$. This is the size of the processed MNIST image after a convolution, by then called the “feature map”. It may be counterintuitive that the convolutional matrix X has as many columns as the “feature map” that exists after the convolution. However, per such feature map pixel or column of X , an entire length of a kernel patch in pixels (times channels if there were more than 1) gets stored. So the matrix X does in fact contain all pixels of the original MNIST image and vastly more. This is the aforementioned memory trade-off.

In addition to the described matrix X , or `im2col_buf` in the implementation, a weight matrix W of size $M \times K$ corresponding to $\text{Output Channels} \times \text{Elements per Kernel Patch}$ and a bias vector B of length M , corresponding to Output Channels , are loaded from the PyTorch training just as in the baseline implementation. Here, matrix W and vector B perform the GEMM together with matrix X :

$$Y = W \cdot X + B.$$

Implementation	Cycles (M)	Time (ms)	vs Base	vs PyTorch
Python/PyTorch	–	0.32	13.94×	1.00×
Baseline Nested-Loop	14.57	4.49	1.00×	0.07×
Restructured Nested-Loop	7.10	2.18	2.05×	0.15×
Hoisted Vars + restrict	7.09	2.17	2.06×	0.15×
Baseline w. blocking	5.96	1.83	2.44×	0.18×
Reordered Nested-Loop	5.30	1.63	2.75×	0.20×
Specialized Implementation	3.32	1.01	4.39×	0.32×
Baseline w. im2col matrix multiplication	1.85	0.57	7.88×	0.57×
NEON Blocked Implementation	1.08	0.33	13.46×	0.97×
Specialized MaxPool2D	1.07	0.33	13.56×	0.98×

Table 2: Performance summary of implementations. Cycle counts (millions) and wall-clock time (ms) are per inference, taken as the median over all runs. *vs Base* and *vs PyTorch* give wall-clock speedups relative to the Baseline Nested-Loop and to the Python/PyTorch reference, respectively (below 1× means slower than PyTorch). The NEON blocked kernel reaches 13.46× over baseline, within 3 % of PyTorch; Specialized MaxPool2D is statistically indistinguishable from it (§4.2.8). All figures are per-inference medians at batch size 1.

As already mentioned for the baseline implementation, in our setup, all containers are flat vectors, so our matrices are not actually represented as 2D arrays in the code. We, again, trace the operations conceptually, but practically work with an offset to operate on rows that are stored consecutively within the flat vector. Specifically, an offset is needed to skip to the next conceptual row during the computation.

An advantage of this is that since the data in `im2col_buf`, for instance, is perfectly contiguous in the flat container, the `#pragma omp simd` directive can be used in the innermost GEMM loop over the columns. This allows the compiler to autovectorize the mathematical operations, which does not violate the strict single-threaded constraints of our comparison.

5 RESULTS

Table 2 collects the measurements. Every C++ variant passed the validation of Section 3 on all 10 000 test images, so the runtimes below all describe implementations that reproduce the PyTorch forward pass to within the stated tolerance. The naive baseline needs 4.49 ms per inference and is 13.94× slower than PyTorch. The fastest result comes from the two NEON-based kernels, which run in 0.33 ms: the NEON blocked convolution reaches 13.46× over the baseline and trails PyTorch by only 1.03×. The Specialized MaxPool2D step, which builds on it, is statistically indistinguishable from it (13.56× versus 13.46× over baseline, a gap within run-to-run noise), so we treat the NEON kernel as the fastest meaningful variant. On a side note: as expected in subsection 3.5, PyTorch performs comparatively better than our implementations at larger batch sizes. An exploratory run at batch size 100 supports this: the NEON kernel that trails PyTorch by 1.03× at batch size 1 fell to 1.71× slower at batch size 100. We did not systematically investigate this within the scope of this report and leave a sweep across batch sizes as a possible extension in Section 6.

5.1 THE CUMULATIVE LADDER

The four steps that preserve the baseline’s full generality recover a factor of 2.75 between them, and they do so very unevenly. Restructuring the loop nest alone accounts for almost all of it (2.05×), reordering the loops adds a further 34 %, and hoisting the index arithmetic together with `restrict` adds essentially nothing: 2.17 ms against 2.18 ms, or 7.09 against 7.10 million cycles, a difference of 0.2 % that is at the level of our measurement noise. This is the clearest negative result in the set. The transformation is the one a reader would expect to help, since it removes redundant multiplications from the innermost loop and promises the compiler that the pointers do not alias, and it is also the one that helped least.

Giving up arbitrary stride and padding takes the speedup from 2.75× to 4.39×. Giving up portability across instruction sets takes it from 4.39× to 13.46×. The hand-written NEON kernel is thus 3.06× faster than the specialized scalar kernel it replaces, against a theoretical ceiling of 4× from the 128-bit vector width, so it recovers roughly 77 % of the available SIMD gain. Both observations point the same way: on this problem the profitable transformations were the ones that changed what the innermost loop does, and the level of specialization at which the gains actually appear is the last one, where the code stops being portable.

The final step, the specialized MaxPool2d of subsection 4.2.8, changes nothing measurable, 0.329 ms against 0.331 ms for the NEON kernel it replaces. This is what Table 1 predicts, since pooling contributes no floating-point work worth naming, and it is a useful confirmation that the measurement is attributing time where we think it is.

5.2 THE TWO SIDE BRANCHES

The blocked baseline reaches 2.44×, slightly better than the first rung of the ladder and well below the reordered variant, so register tiling for reuse was worth less here than simply fixing the loop order. That is consistent

with the working set analysis: at batch size 1 the whole 517 KiB fits in private L2, so there is little bandwidth pressure for tiling to relieve.

The im2col variant is the more interesting one. At 0.57 ms it is $7.88\times$ faster than the baseline and beats every rung of the ladder except the final two, NEON-based steps, while remaining as general as the baseline and portable to any target the compiler supports. It gets there by changing the algorithm rather than the instruction mix: once the convolution is laid out as a GEMM, the innermost loop is a contiguous dot product that `#pragma omp simd` vectorizes without us writing a single intrinsic. Against the roughly $2\times$ memory cost of the im2col_buf patch matrix, this is the best ratio of speedup to lost generality in the whole set.

6 CONCLUSION AND OUTLOOK

We initially asked at which level of optimization the gains would appear, and for this problem they appeared at a higher level than expected. Everything that preserves the baseline’s generality was worth $2.75\times$ together, fixing stride and padding bought another $1.6\times$, and the NEON kernel, the one step that tied the code to a single instruction set, bought a further $3.06\times$ on its own. Hoisting index arithmetic and adding restrict were worth nothing measurable. The exception was im2col: recasting the convolution as a GEMM reached $7.88\times$ while staying portable and fully general. Changing the algorithm so the compiler can vectorize on our behalf paid nearly as well as intrinsics at none of their cost in portability.

A single caveat qualifies all of this. Our best variant sustains about 18.7 GFLOP/s against a single-core peak near 105.6 GFLOP/s, roughly 17.7% of peak, so a meaningful share of the machine’s capability is reached, though most of it still goes unused.

The setup for our CNN forward pass was deliberately kept within narrow bounds. We identified a few potential pathways for generalizing the implementation to enable further optimizations. The detailed ideas that could make use of the aforementioned remaining machine capability are listed in the following.

- Vary the architecture, especially the input dimensions to the convolutional layers. Every conclusion here is drawn at one size that fits in L2, the regime that most favors specialization; larger inputs should change the ranking.
- Vary the batch size upward from the batch-1 measurements reported here, where fixed per-call costs such as weight packing are least amortized, to see how the ranking shifts as the vector units get more contiguous work.
- Compare against a PyTorch build with oneDNN and ACL enabled, and extend the vectorized kernel from NEON to SVE.
- Autotune OC_TILE and OW_TILE, fixed at 4 by hand and not optimized, and place the measurements on a roofline model.

7 ACKNOWLEDGMENTS

We would like to thank the HPC4WC team for organizing the course and supporting us during the project. We used generative AI as assistants for code implementations, debugging and report writing. The creative process and possible mistakes are our own.

REFERENCES

- [1] Keiron O’Shea and Ryan Nash. An introduction to convolutional neural networks, 2015. URL <https://arxiv.org/abs/1511.08458>.
- [2] Zijie J. Wang, Robert Turko, Omar Shaikh, Haekyu Park, Nilaksh Das, Fred Hohman, Minsuk Kahng, and Duen Horng Polo Chau. Cnn explainer: Learning convolutional neural networks with interactive visualization. *IEEE Transactions on Visualization and Computer Graphics*, 27(2): 1396–1406, 2021. doi: 10.1109/TVCG.2020.3030418.
- [3] Nick Brown, Maurice Jamieson, Anton Lydike, Emilien Bauer, and Tobias Grosser. Fortran performance optimisation and auto-parallelisation by leveraging mlir-based domain specific abstractions in flang. In *Proceedings of the SC ’23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis*, SC-W 2023, pp. 904–913. ACM, November 2023. doi: 10.1145/3624062.3624167. URL <http://dx.doi.org/10.1145/3624062.3624167>.
- [4] John L. Hennessy and David A. Patterson. A new golden age for computer architecture. *Commun. ACM*, 62(2):48–60, January 2019. ISSN 0001-0782. doi: 10.1145/3282307. URL <https://doi.org/10.1145/3282307>.
- [5] Enrique G. Paredes, Linus Groner, Stefano Ubiali, Hannes Vogt, Alberto Madonna, Kean Mariotti, Felipe Cruz, Lucas Benedicic, Mauro Bianco, Joost VandeVondele, and Thomas C. Schulthess. Gt4py: High performance stencils for weather and climate applications using python, 2023. URL <https://arxiv.org/abs/2311.08322>.
- [6] Tobias Gysi, Christoph Müller, Oleksandr Zinenko, Stephan Herhut, Eddie Davis, Tobias Wicky, Oliver Fuhrer, Torsten Hoefer, and Tobias Grosser. Domain-specific multi-level ir rewriting for gpu, 2020. URL <https://arxiv.org/abs/2005.13014>.
- [7] Jack Kosaian and K. V. Rashmi. Arithmetic-intensity-guided fault tolerance for neural network inference on gpus. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC ’21, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450384421. doi: 10.1145/3458817.3476184. URL <https://doi.org/10.1145/3458817.3476184>.

- [8] Wouter Dekeyser, Xavier Bonnin, Steven W. Lisgo, Richard A. Pitts, and Brian LaBombard. Implementation of a 9-point stencil in solps-iter and implications for alcator c-mod divertor plasma simulations. *Nuclear Materials and Energy*, 18: 125–130, 2019. ISSN 2352-1791. doi: <https://doi.org/10.1016/j.nme.2018.12.016>. URL <https://www.sciencedirect.com/science/article/pii/S2352179118302011>.
- [9] Nisarg Ujjainkar, Jingwen Leng, and Yuhao Zhu. Imagen: A general framework for generating memory- and power-efficient image processing accelerators. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700958. doi: 10.1145/3579371.3589076. URL <https://doi.org/10.1145/3579371.3589076>.
- [10] F.D. Witherden, A.M. Farrington, and P.E. Vincent. Pyfr: An open source framework for solving advection–diffusion type problems on streaming architectures using the flux reconstruction approach. *Computer Physics Communications*, 185 (11):3028–3040, 2014. ISSN 0010-4655. doi: <https://doi.org/10.1016/j.cpc.2014.07.011>. URL <https://www.sciencedirect.com/science/article/pii/S0010465514002549>.
- [11] Alain Denzler, Geraldo F. Oliveira, Nastaran Hajinazar, Rahul Bera, Gagandeep Singh, Juan Gómez-Luna, and Onur Mutlu. Casper: Accelerating stencil computations using near-cache processing. *IEEE Access*, 11:22136–22154, 2023. doi: 10.1109/ACCESS.2023.3252002.
- [12] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- [13] PyTorch. MNIST — Torchvision main documentation. <https://docs.pytorch.org/vision/0.24/generated/torchvision.datasets.MNIST.html#torchvision.datasets.MNIST>, 2025. Accessed: 2026-07-31.
- [14] Chhavi Yadav and Leon Bottou. Cold case: The lost mnist digits. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/51c68dc084cb0b8467eafad1330bce66-Paper.pdf.
- [15] Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. *International Joint Conference on Artificial Intelligence (IJCAI)*, 1995.
- [16] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL <https://arxiv.org/abs/1412.6980>.
- [17] Swiss National Supercomputing Centre (CSCS). Santis cluster documentation. <https://docs.cscs.ch/clusters/santis/>, n.d. URL <https://docs.cscs.ch/clusters/santis/>. Accessed: 2026-08-01.
- [18] NVIDIA Corporation. NVIDIA Grace Hopper Superchip Architecture Whitepaper. Technical report, NVIDIA Corporation, 2024. URL <https://dam-cdn.nvd.orangelogic.com/AssetLink/h125m07nuew832vthbov0q6b2rs8ta0q.pdf>. Version 1.21, Accessible via: <https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/>.
- [19] Arm Limited. Arm neoverse v2 core software optimization guide. Technical Report 109898, Arm Limited, 2022. URL <https://developer.arm.com/documentation/109898/latest/>. Document ID PJDOC-466751330-593177, Issue 3.0.
- [20] Meta. Announcing the PyTorch Foundation to accelerate progress in AI research, Sep 2022. URL <https://about.fb.com/news/2022/09/pytorch-foundation-to-accelerate-progress-in-ai-research/>. Accessed: 2026-08-05.
- [21] Adam Paszke et al. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bf9f7012727740-Paper.pdf.
- [22] PyTorch Contributors. Autograd mechanics — PyTorch documentation, 2017. URL <https://docs.pytorch.org/docs/stable/notes/autograd.html>. Accessed: 2026-08-05.
- [23] PyTorch. ATen: Tensor library. <https://docs.pytorch.org/cppdocs/api/aten/index.html>, n.d. PyTorch C++ documentation, accessed 2026-08-25.
- [24] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL <http://www.deeplearningbook.org>.
- [25] Kumar Chellapilla, Sidd Puri, and Patrice Simard. High performance convolutional neural networks for document processing. In *Tenth International Workshop on Frontiers in Handwriting Recognition*. Université de Rennes, 2006. URL <https://inria.hal.science/inria-00112631v1>.